

AGENTES IA

Master em Agentes RAG & Inteligência Artificial Autônoma

Aprenda a construir agentes inteligentes autônomos com LangChain,
bancos de dados vetoriais (Pinecone) e modelos LLM conectados a
APIs corporativas.

[**Nível: Avançado**]

[**Duração: 18h**]

[**12 Módulos**]

[**Tech: Python, LangChain, Pinecone, FastAPI**]

#> Módulo 1: Arquitetura de Agentes e LLMs

Bem-vindo ao nível avançado de Inteligência Artificial. Até agora, você provavelmente usou o ChatGPT como um oráculo: você faz uma pergunta e ele dá uma resposta. No entanto, em ambientes corporativos, isso não é suficiente. Queremos que a IA atue por si mesma.

O que é um Agente Autônomo?

Um agente é um sistema onde o grande modelo de linguagem (LLM) funciona como o 'cérebro' ou motor de raciocínio de um aplicativo. O LLM decide em qual ordem executar as tarefas, quais ferramentas usar (como buscar no Google ou consultar um banco de dados) e quando terminou a tarefa.

Preparando seu ambiente

Para começar, você precisa de uma conta na OpenAI (platform.openai.com) para obter uma API Key, já que usaremos o GPT-4o como nosso motor de raciocínio. Além disso, instalaremos o ecossistema LangChain.

```
pip install langchain langchain-openai python-dotenv

# Crie um arquivo chamado .env em seu projeto e adicione:
OPENAI_API_KEY="sk-sua-chave-secreta"
LANGCHAIN_API_KEY="lsv2_sua-chave"
```

O uso do arquivo ``.env`` é uma prática padrão de segurança cibernética para evitar enviar senhas para o GitHub. Usaremos a biblioteca `python-dotenv` para carregar essas variáveis em nosso script automaticamente.

#> Módulo 2: Aprofundando no LangChain

LangChain é o framework padrão da indústria para construir aplicativos impulsionados por LLMs. Seu conceito principal é a 'Cadeia' (Chain), que permite sequenciar componentes.

Componentes Principais (LCEL)

A Linguagem de Expressão do LangChain (LCEL) nos permite usar o símbolo '|' (pipe) para conectar um Prompt, um Modelo e um Parser, semelhante a como os terminais Linux funcionam.

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv() # Carrega sua API KEY

# 1. Definimos o modelo
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

# 2. Definimos o modelo de prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é um assistente sarcástico, mas útil."),
    ("user", "Explique o que é {tema}")
])

# 3. Definimos o Output Parser
parser = StrOutputParser()

# Criamos a Cadeia
cadeia = prompt | llm | parser

# Invocamos a cadeia
resultado = cadeia.invoke({"tema": "Machine Learning"})
print(resultado)
```

Explicação:

- Temperature (0.2): Controla a criatividade. Um valor baixo (0) torna o modelo mais analítico. Um valor alto (1) o torna mais criativo.
- Output Parser: Normalmente, o modelo retorna um objeto complexo. O

#> Módulo 3: Embeddings e Representação Semântica

Os LLMs não 'lêem' texto como os humanos. Eles operam com matemática. Para que a IA possa pesquisar documentos com base em seu 'significado' (pesquisa semântica), devemos converter o texto em vetores.

O Espaço Vetorial

Um Embedding é um vetor matemático (uma lista de milhares de números) que representa o significado de um texto. Textos com significado semelhante estarão 'perto' matematicamente neste espaço multidimensional.

```
from langchain_openai import OpenAIEmbeddings

# Inicializamos o modelo gerador de embeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

texto_1 = "O cachorro brinca no parque"
texto_2 = "Um canino se diverte na praça"

# Convertemos o texto em números
vetor_1 = embeddings.embed_query(texto_1)
vetor_2 = embeddings.embed_query(texto_2)

print(f"Dimensões do vetor: {len(vetor_1)}")
print("Primeiros 5 valores:", vetor_1[:5])
```

Embora as palavras em 'texto_1' e 'texto_2' sejam diferentes, seus vetores resultantes serão matematicamente muito semelhantes, permitindo que a IA saiba que estão falando da mesma coisa.

#> Módulo 4: Bancos de Dados Vetoriais (Pinecone)

Um banco de dados SQL tradicional (como MySQL) não serve para armazenar vetores de 1500 dimensões e buscar semelhanças rapidamente. Para isso usamos Pinecone, um Banco de Dados Vetorial especializado em milissegundos.

Conectando e Inserindo Dados

Primeiro, você precisa instalar o SDK: ``pip install pinecone-client langchain-pinecone`` e configurar ``PINECONE_API_KEY`` no seu ``.env``.

```
import os
from pinecone import Pinecone, ServerlessSpec
from langchain_pinecone import PineconeVectorStore

pc = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
index_name = "curso-rag"

# Criar o índice se não existir
if index_name not in pc.list_indexes().names():
    pc.create_index(
        name=index_name,
        dimension=1536, # Dimensão do modelo OpenAI
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

# Inserir documentos reais
textos = [
    "As políticas de reembolso da empresa exigem 30 dias.",
    "O horário do suporte técnico é das 9h às 18h EST."
]

vectorstore = PineconeVectorStore.from_texts(
    textos,
    embeddings,
    index_name=index_name
)
```

Explicação:

#> Módulo 5: Arquitetura RAG Completa

RAG (Retrieval-Augmented Generation) resolve o problema de 'alucinações' do LLM. Em vez de depender do que o modelo memorizou durante o treinamento, damos a ele o documento exato ANTES de ele responder.

Fluxo RAG de Ponta a Ponta

1. Recuperador (Retriever): Busca os 3 documentos mais relevantes à pergunta do usuário no Pinecone.
2. Injeção de Contexto: Cola esses documentos no Prompt.
3. Geração: O LLM lê o prompt com o contexto e responde.

```
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain.chains import create_retrieval_chain

# O retriever irá buscar no Pinecone
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

prompt = ChatPromptTemplate.from_template("""
Responda à pergunta com base APENAS no seguinte contexto:

<contexto>
{context}
</contexto>

Pergunta: {input}
""")

# Criamos a cadeia que injeta documentos
document_chain = create_stuff_documents_chain(llm, prompt)

# Unimos o recuperador com a cadeia
rag_chain = create_retrieval_chain(retriever, document_chain)

resposta = rag_chain.invoke({"input": "Qual é o horário de suporte?"})
print(resposta["answer"])
```

Graças a isso, o LLM responderá 'Das 9h às 18h EST' com base nos dados corporativos seguros.

#> Módulo 6: Tools e Conexão a APIs

O RAG nos permite ler documentos estáticos. Mas um Agente de IA real precisa escrever e-mails, consultar bancos de dados SQL, verificar o clima ou preços de ações ao vivo. Isso é alcançado por meio de 'Ferramentas' (Tools).

Definindo Ferramentas para o Modelo

Uma ferramenta no LangChain é uma função Python decorada, que obrigatoriamente requer docstrings claros, porque o LLM os lê para entender para que serve a ferramenta.

```
from langchain_core.tools import tool
import requests

@tool
def consultar_clima(cidade: str) -> str:
    """Utilize esta ferramenta EXCLUSIVAMENTE para obter o clima atual de uma cidade especificada."""
    # Simulação de uma chamada de API REST:
    return f"O clima em {cidade} é de 24°C, parcialmente nublado."

@tool
def calculadora(expressao: str) -> str:
    """Resolve expressões matemáticas complexas."""
    return str(eval(expressao))

lista_ferramentas = [consultar_clima, calculadora]
```

Agora temos funções Python que o modelo pode decidir usar por conta própria.

#> Módulo 7: O Framework ReAct (Reasoning and Acting)

ReAct é o padrão de design onde a IA pensa passo a passo. Uma pergunta é feita, a IA 'Pensa' sobre qual ferramenta precisa usar, 'Age' invocando a ferramenta e 'Observa' o resultado. Repita o ciclo até ter a resposta final.

Implementação do Agente Autônomo

Combinaremos nossas ferramentas com o modelo.

```
from langchain.agents import create_tool_calling_agent, AgentExecutor

# Forçamos o modelo a usar as ferramentas
llm_com_tools = llm.bind_tools(lista_ferramentas)

prompt_agente = ChatPromptTemplate.from_messages([
    ("system", "Você é um poderoso assistente de IA autônomo."),
    ("user", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

agente = create_tool_calling_agent(llm, lista_ferramentas, prompt_agente)

# O Executor é o motor que roda o loop while infinito do ReAct
executor = AgentExecutor(agent=agente, tools=lista_ferramentas, verbose=True)

resposta = executor.invoke({"input": "Qual é a raiz quadrada de 144 e como está o tempo em Paris?"})
print(resposta['output'])
```

Explicação:

- `verbose=True` é essencial: mostra no terminal cada pensamento e ação que o modelo toma em tempo real.
- `agent\_scratchpad` é a memória de curto prazo onde o agente salva as etapas que já tentou.

#> Módulo 8: Memória e Contexto de Longo Prazo

Um agente sem memória sofre de amnésia a cada nova mensagem. Em sistemas de IA, a memória é obtida enviando todo o histórico de bate-papo anterior em cada nova chamada ao modelo.

Tipos de Memória

Existem memórias de Buffer (salvam tudo) e memórias de Resumo (usam outro LLM pequeno para resumir bate-papos antigos e economizar tokens). Usaremos BufferMemory.

```
from langchain.memory import ConversationBufferMemory

# Criamos uma memória que guarda mensagens
memoria = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

# Modificamos o prompt para incluir o histórico:
prompt_com_memoria = ChatPromptTemplate.from_messages([
    ("system", "Você é um assistente AI."),
    ("placeholder", "{chat_history}"), # Injeção dinâmica de memória
    ("user", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

# Reconstruímos o agente injetando a memória no executor...
executor_memorioso = AgentExecutor(
    agent=agente,
    tools=lista_ferramentas,
    memory=memoria,
    verbose=True
)
```

#> Módulo 9: Sistemas Multi-Agente com LangGraph

Os casos de uso corporativos (como programar código, revisar vulnerabilidades e compilar) são muito complexos para um único agente. LangGraph permite criar fluxos de trabalho (Grafos) com múltiplos agentes especializados.

Fluxo de Nós

No LangGraph, você define um 'Estado' (um dicionário). Cada nó (Agente A, Agente B) recebe o estado, o modifica e o passa para o próximo nó. A lógica de roteamento é definida com Edge Conditions.

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator

class EstadoAgente(TypedDict):
    mensagens: Annotated[list, operator.add]

def no_investigador(estado):
    # Lógica do agente investigador...
    return {"mensagens": ["Encontrei esta informação."]}

def no_redator(estado):
    # Lógica do agente redator...
    return {"mensagens": ["Escrevendo relatório..."]}

grafo = StateGraph(EstadoAgente)
grafo.add_node("Investigador", no_investigador)
grafo.add_node("Redator", no_redator)

# Fluxo
grafo.add_edge(START, "Investigador")
grafo.add_edge("Investigador", "Redator")
grafo.add_edge("Redator", END)

app = grafo.compile()
```

Este grafo determinístico nos permite construir processos de negócios

#> Módulo 10: Desenvolvimento de Backend com FastAPI

Um script no seu computador não pode ser usado por milhares de clientes. Precisamos criar um Backend robusto (API REST) que sirva seu Agente para qualquer front-end (React, Vue, Android).

Expondo o Agente para o Mundo

Instalamos o FastAPI e o servidor Uvicorn: ``pip install fastapi uvicorn pydantic``

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Servidor de Agentes IA")

# Modelo de validação de dados de entrada
class RequisicaoUsuario(BaseModel):
    session_id: str
    mensagem: str

@app.post("/api/v1/chat")
async def conversar(requisicao: RequisicaoUsuario):
    try:
        # Invocamos o agente que criamos nos módulos anteriores
        resposta = executor.invoke({"input": requisicao.mensagem})
        return {
            "status": "success",
            "resposta_ia": resposta["output"]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# Para rodar o servidor, use no seu terminal:
# uvicorn arquivo:app --reload
```

#> Módulo 11: Implantação e Monitoramento

Lançar IA na produção é perigoso se você não souber o que o modelo está fazendo internamente. LangSmith é o painel oficial da LangChain para rastreabilidade.

LangSmith Tracing

Adicionando 3 variáveis ao seu `.env`, todo o tráfego, uso de tokens, latências e erros de seus agentes serão salvos em um painel ao vivo na nuvem.

```
LANGCHAIN_TRACING_V2="true"
LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"
LANGCHAIN_API_KEY="lsv2_sua_chave_smith"
LANGCHAIN_PROJECT="agente_producao_v1"
```

Uma vez que o agente é validado, usamos Docker para empacotar o código FastAPI, suas dependências Python e variáveis de ambiente em um contêiner isolado, pronto para AWS ou Google Cloud.

#> Módulo 12: Projeto Final (Agente Corporativo)

O exame final. Você combinará TUDO o que aprendeu. Você vai desenvolver um 'Representante de Vendas Autônomo'.

Arquitetura Necessária

1. Embeddings & RAG: Você ingerirá 50 PDFs de catálogos de produtos no Pinecone.
2. Tools de Banco de Dados: Desenvolverá uma ferramenta SQLTool para o agente verificar o estoque ao vivo de um banco MySQL.
3. Framework ReAct: O agente deve deduzir o produto que o cliente deseja, pesquisar detalhes no RAG, verificar o estoque e calcular o preço com desconto.
4. Backend: Tudo montado no FastAPI com rotas seguras.

Ao concluir isso, você oficialmente será um Arquiteto de Inteligência Artificial! O mercado corporativo está procurando esse nível de expertise globalmente.

MASTER EM AGENTES IA & RAG

Certificação concedida a:

[ARCHITECT_NAME]

Por ter dominado o design, desenvolvimento e implantação de Sistemas Multi-Agentes Autônomos, integrando arquiteturas RAG, Bancos de Dados Vetoriais e desenvolvimento Backend.

TIMESTAMP: 2026-08-12 // AI_MASTER_NODE_SIGNATURE